# Additional Tasks
## *Object-Oriented Design, IV1350*

Ellen Grönholm
ellengro@kth.se

VT2026 v22

## Declaration:

By submitting this assignment, it is hereby declared that all group members listed above have contributed to the solution. It is also declared that all project members fully understand all parts of the final solution and can explain it upon request.

It is furthermore declared that no part of the solution has been copied from any other source (except for resources presented in the Canvas page for the course IV1350), and that no part of the solution has been provided by someone not listed as a project member above.

## Usage of AI

The usage of AI assistance in the report is limited to help of environmental issues with the IDE and explaining observed exceptions when coding. As AI assistance ChatGPT-5.5 (OpenAI) and Gemini 3 Flash (Google) have been used.

# 1 Introduction

The additional tasks build directly upon the "Repair Electric Bike" program developed across seminars one to four by refining the architecture and verifying the output quality of the code. Task 1 enhances the Observer pattern implementation from Seminar 4 by introducing a template class to handle the update logic and exception management for the `RepairOrderView` and `RepairOrderLogger`. Task 2 uses the foundational knowledge of inheritance and encapsulation gained during designing to explore the trade-offs between inheritance and composition using classes from the standard Java libraries. Finally, Task 3 increases the robustness of the system by ensuring that all user-facing information, such as the diagnostic reports and total costs presented to the customer, is verified through JUnit testing. All in all, these tasks ensure the final system is not only functional but also sticks to object-oriented principles and testing standards.

# 2 Method

## 2.1 Task 1: Template Method Pattern

The *Template Method* design was used to expand on observer design pattern. From the instructions a skeleton code was received of how to structure the template. This template restructured how to handle updates to repair orders by having code to be executed when the order was updated, and then having a part that handles any exceptions thrown.

To implement the template, these two parts first had to be found within the observer classes, which was harder said than done. This was due to the structure of my code, not originally throwing any exceptions. The template, however, incited the realization that unchecked exceptions might occur as data might be missing from the DTO, or that there might be issues with printing exceptions to files. This was thus handled due to the template.

Then both observer codes were observed further, as both observers used the same structure:

```
if(repOrderDTO.getStateOfOrder() == StateOfOrder.MECHANIC_REVIEW){
    //Do something (print to file or terminal)
} else if(repOrderDTO.getStateOfOrder() == StateOfOrder.MECHANIC_COMPLETED){
    //Do something (print to file or terminal)
}
```

Since this code existed in both classes, this should instead be part of the template. The template was thus restructured to contain two new methods `orderIsUpdatedForMechanic()` and `orderIsUpdatedForClient()`. So that the outline the `ifelse` could be in the template:

```
if(repOrderDTO.getStateOfOrder() == StateOfOrder.MECHANIC_REVIEW){
    orderIsUpdatedForMechanic(repOrderDTO);
} else if(repOrderDTO.getStateOfOrder() == StateOfOrder.MECHANIC_COMPLETED){
    orderIsUpdatedForClient(repOrderDTO);
}
```

## 2.2 Task 2: Inheritance vs. Composition

The thought was to implement a class that extends `java.util.random` by generating a random dicount. This might be used in the shop during promotions or if a customer has had a negative experience. However, instead of implementing a `Random` directly, two limiting factors are put on it. Partly, a maximum value needs to be submitted when applying the discount. This makes it possible to adapt the max-range of the discount depending on campaigns or the severity of the customer dissatisfaction. The limit is hard coded, but since it is not expected that the random discount should be used often, the thought is that for campaign launches, the developers would change this maximum value, and then put it to 0 or the 'customer dissatisfaction level'. Secondary, there is a maximum limit of how many times the `randomDiscount` can be called since in this scenario it is believed that the higher ups would not be pleased if too many of these 'random discounts' were handed away.

The selection of the `java.util.Random` class for this adaptation is based on the specific recommendations found in the *additional higher-grade tasks* documentation, which suggests this utility class as a suitable candidate for comparing different design structures. The thinking behind this mainly strove from that this choice adheres to the strict requirement that explicitly "omits any List types" from the adaptation task.

The workflow for creating the two distinct codes follow a systematic progression from direct extension to modular wrapping. For the class inheritance method a class `DiscountRandom` was created and defined the new class as a direct subclass of `java.util.Random`. The method to adapt was chosen to become `nextInt(int bound)` and overrides it to incorporate new domain logic, such as a private counter that enforces a maximum usage limit.

In circumvent this, the workflow for the object composition method prioritizes a restricted and well-defined public interface. Instead of becoming a child of the library class, a standalone class `DiscountGenerator` is created that maintains a private instance of `java.util.Random` as an internal field. This way, the code doesn't automatically expose any of the library's methods; instead only the specific public methods required for the system's needs are manually chosen and written.

### 2.3 Task 3: Testing Output

The code contains two types of printable messages. The 'hardcoded view' and the program specific output. After discussing with Niharika, it was decided that the task 3 regarded program specific output only. The code that should be tested is thus not the informational message "*the receptionist added the problem 'TIRES'*" as this is supposed to be part of a more advanced view. The code that should be tested is the code that is part of the system, the code that the users, both receptionists and mechanics as well as developers should see.

During the implementation of the code for seminar three and four, most of the printable code had already been tested. Most of the outputting strings, are created lower in the code. The class `PrintToString` converts the receipt to a string, which is then used by the repair order via the class `PrintToFile`. As the code is structured as this, the receipt will be returned as a string by methods which makes it easy to be tested. The receipt as a string can thus be tested both in the class where it is created, as well as in all classes above which use it.

The other printable output that is part of the system are the error messages, both ones printed in the terminal as well as in files. When testing this, it was mostly structured to print all exceptions to a file, and then read from the file if the error messages match the expected messages. Other ways to ensure the correct structure of error messages, was to catch the messages and then measure that the messages equal the expected output.

The instructions enforce that the unit tests must cover:

- The `main` method.
- The `View` class, specifically all printouts that contain actual information.
- The final repair order printout.
- Any other methods in your program that utilize `System.out`.

but as noted, these classes and methods had already been tested before based on the criteria from seminar 4; that it is Self-Evaluating, contains Branch Coverage and that the test classes are located in the same package as the class but in a separate directory. The only change is that the programs main function does not print anything to the terminal, only the view does. It is thus impossible to test this.

## 3 Result

This section presents the structural outcomes, source code explanations, and verification of execution.

## 3.1 Git Repository

https://gits-15.sys.kth.se/ellengro/IV1350-ObjektOrienteradDesign

## 3.2 Task 1: Template Method Pattern Implementation

The template was, as seen in 2.1, restructured to adapt better the the already existing code:

```
if(repOrderDTO.getStateOfOrder() == StateOfOrder.MECHANIC_REVIEW){
    orderIsUpdatedForMechanic(repOrderDTO);
} else if(repOrderDTO.getStateOfOrder() == StateOfOrder.MECHANIC_COMPLETED){
    orderIsUpdatedForClient(repOrderDTO);
}
```

this code is then surrounded by a `try-catch` block. The code in the observers then implemented the template class and contained the methods `orderIsUpdatedForMechanic` and `orderIsUpdatedForClient` as well as a method for exception handling `handleErrors`. The the observing class `RepairOrderLogger`, the code went from containing one method `repairOrderIsUpdated`, which was the original method enforced by the observer pattern, to now contain the three above mentioned methods. The `orderIsUpdatedForMechanic` for example, was now updated to look like:

```
FileLogger logger = new FileLogger("MechanicReview");
logger.log("Hi Mechanic, there is a new order to be reviewed: \n" +
            //repOrderDTO.getRepairOrderAsString());
```

for the `RepairOrderView` the same method `orderIsUpdatedForMechanic` instead looks like:

```
System.out.println("Hi Mechanic, there is a new order to be reviewed: \n" +
            //repOrderDTO.getRepairOrderAsString());
```

The Template Method thus defines a fixed sequence of steps in a base template class that all observers must follow. When a repair order is updated, the observer interface method calls a private method that contains the core logic. This ensures that every observer, whether it is printing to the console or logging to a file, executes its logic within a predefined structure, thus standardizing error handling and simplifying the structure.

### 3.3 Task 2: Inheritance vs. Composition Implementation

The inheritance based code was implemented by creating a new class that `extends Random` and `implements DiscountStrategy`. As it implements `DiscountSTrategy`, the method `priceAfterDiscount` must be present. This method then calls on the Random methods, however, the 'random-method' is not called from `java.Util.Random`, it is instead defined in this class as `public int nextInt(int bound)` using `@Override`. The whole code looks like:

```
@Override
public int nextInt(int bound) {
    if (currentUsages >= maxUsages) {
        return 0;
    }
    currentUsages++;
    return super.nextInt(bound);
}

@Override
public int priceAfterDiscount(int priceBefore) {
    int discountPercent = nextInt(51); //Up to 50%
    double discountMultiplier = (100.0 - discountPercent) / 100.0;

    return (int) Math.ceil(discountMultiplier*priceBefore);
}
```

The `Random` class is thus extended to limit the `nextInt` method to only be called using a bound. And at the same time it limits how often the the method can be called.

The composition based code is implemented in a new class `DiscountGenerator` and is structured as following:

```
private final Random randomSource = new Random();
private final int maxUsages;
private int currentUsages = 0;

@Override
public int priceAfterDiscount(int priceBefore) {
    if (currentUsages >= maxUsages) {
        return priceBefore;
    }
    currentUsages++;
```

```
        int discountPercent = randomSource.nextInt(51);
        double discountMultiplier = (100.0 - discountPercent) / 100.0;

        return (int) Math.ceil(priceBefore * discountMultiplier);
    }
```

By specifying the Random as a private final it cannot be changed by external classes. Then the internal logic is just implemented within the `priceAfterDiscount` method, thus limiting the discount amount and usage times.

The first approach thus utilizes class inheritance, where a new class is created by directly extending the library class. In this version, the class incorporates a private counter to track how many times a random value has been generated and overrides the specific method `nextInt(int bound)`, to enforce a maximum usage limit. The second approach, on the other hand, employs object composition by creating a standalone class that wraps a private instance of the library class as an internal field. Instead of inheriting from `java.util.Random`, this code manages its own state and provides a specific public method that delegates the generation of the random value to the contained instance.

By implementing the `DiscountSTrategy` theses two discount implementations are called just like any other discount. They are changed "on" or "off" via the method `setDiscount(DiscountStrategy strategy)` which is employed down the layers to the Task lists. Where the `setTotalCost()` finally applies the discount. The code does, due to being integrated into a functioning system, not display how the inheritance could be bypassed. This was as it seemed counterproductive to implement it. However if this wants to be proven, the following main can be added to the code and be executed seperatly:

```
    public static void main(String[] args) {
        InheritanceLimitedRandom generator = new InheritanceLimitedRandom(2);

        System.out.println("--- Using the Limited/Overridden Method ---");
        System.out.println("Roll 1 (nextInt): " + generator.nextInt(100));
        System.out.println("Roll 2 (nextInt): " + generator.nextInt(100));
        System.out.println("Roll 3 (Limit reached, returns 0): " +
                           generator.nextInt(100));

        System.out.println("\n- The Inheritance Flaw: Exposing the Superclass -");
        System.out.println("Bypassing limit via nextLong(): "  +
                           generator.nextLong());
        System.out.println("Bypassing limit via nextDouble(): " +
                           generator.nextDouble());
```

```
        generator.setSeed(12345L);
        System.out.println("Superclass state modified via setSeed().");
    }
```

## 3.4 Task 3: Testing Output Implementation

Most printed material is the receipt at its different stages. That the receipt is printed correctly is thus well established. For this the code looks like the following:

```
    @Test
    public void recieptFaultListingsInfoIsCorrect(){
        String orderRecipt = "";
        orderRecipt += printToReccipt.showReceipt(repOrder);
        assertTrue(orderRecipt.contains("Problem Description: \n" + //
                        " ------------------------------\n" + //
                        "Current bike faults: \n" + //
                        " - BRAKES\n" + //
                        " - CHAIN\n" + //
                        "\n\n" + //
                        "Diagnostic Report: \n" + //
                        " ------------------------------\n" + //
                        "Current bike faults: \n" + //
                        " - BRAKES\n" + //
                        " - CHAIN\n" + //
                        " - PEDALS\n" + //
                        "\n\n")
                        , "The some of the faults are not correctly" +//
                        "recorded on the reciept.");

    }
```

where all of the different parts are tested separately. So `@BeforeEach` a complete repair order is created, and then it is ensured that the receipt contains the information it should. Since the receipt is printed at different stages or the order, it is also tested that the receipt doesn't contain excess information before it should.

The second type of output tested repeatedly is the exception handling. This is general done as either:

```
        @Test
    public void noRepOrderFoundHandled() {
        try {
```

```
            repOrderReg.addRepairOrder(customer);
            repOrderReg.getRepairOrder(13);
            fail("An error was not thrown when no rep order is found");
        } catch (ObjectNotFoundException exc) {
            assertNotNull(exc, "No exception is thrown");
            assertEquals("Repair Order", exc.getObjectName(),
            "The error message is for the wrong database");
            assertEquals("13", exc.getWrongData(),
            "The error message is for the wrong database");
            assertTrue(exc.getErrorMessage().contains(
            "The repair order id was not found in the database."),
            "The error message is for the wrong");
        } catch (DataBaseConnectionException exc) {
            fail("An DataBaseConnection error was thrown when it shouldn't");
        } catch (EmptyRegistryException exc) {
            fail("An EmptyRegistry error was thrown when it shouldn't");
        }
    }
```

or:

```
        try {
            File Obj = new File("TestController" + identifyingTime + ".txt");
            Scanner Reader = new Scanner(Obj);

            String allLogged = "";
            // Traversing File Data
                  while (Reader.hasNextLine()) {
                String data = Reader.nextLine();
                allLogged += data;
            }
            Reader.close();
            assertTrue(allLogged.contains(loggerString),
            "The correct text is not Logged");
        } catch (Exception exc) {
            fail("The file could not be found." + //
            "The logger is thus not changed to FileLogger");
        }
```

The code thus ensures that error messages are correct by intercepting the exceptions thrown and double checking that their messages are correct. At the final stage it is also checked by moving the exception to a file and by then reading the thrown exception from

the file. The code also checks that exceptions are not thrown when they shouldn't by asserting `fail("...")` if these catches are reached.

# 4 Discussion

## 4.1 Task 1: Evaluation of the Template Method Pattern

The Template Method pattern betters the observer implementations by providing a standardized "skeleton" for how updates are processed. This ensures consistent behavior and forced mandatory error handling across all observers.

One of the primary enhancements is the mandatory inclusion of a `try-catch` block within the template. The template class implements the error-handling logic only once, ensuring that specific implementations do not forget to handle potential exceptions by requiring subclasses to implement an abstract `handleErrors(Exception e)` method, forcing them to define how they will respond to failures while the template maintains control over the overall flow. By using the Template Method, the observers are further decoupled from the control flow logic since the Template Class handles the high-level coordination of the update while the subclasses only need to implement the methods, focusing purely on their specific task, such as formatting the repair order data for the display.

This pattern prevents code duplication by centralizing the update-handling logic which betters cohesion. By allowing the concrete observer classes to focus exclusively on their single responsibility, either printing to the console or writing to a file, while the superclass handles the secondary responsibility of flow control and error management, cohesion is further bettered. If the way the system processes observer updates needs to change (for example, adding a logging timestamp before every update), it only needs to be modified in the template class rather than in every individual observer implementation.

By using inheritance to share this logic, however, the observers are locked into a rigid hierarchy. As noted in the sources, inheritance is often considered a "second reason" to prefer composition because it can break encapsulation. If a future requirement demands an observer that does not follow this specific try-catch flow, it cannot easily be integrated into this hierarchy. This pattern thus creates a fragile base class; any change to the super class's `handleRepairOrderUpdate` method (like adding a new mandatory step in the algorithm) will automatically impact all subclasses. This can lead to unexpected side effects across the entire program if the base template is modified without full consideration of every observer's needs.

A factor that was struggled with was thus the decision of how much to add to the template and how much to keep within the extensions of the template. Both extensions contain a comparison of in what instance they are called depending on who the update is aimed for. Following the pseudocode obtained for the task, this cannot be handled

correctly. It was thus chosen to remove the method `doHandleRepairOrderUpdate` and instead add two methods `orderIsUpdatedForMechanic` and `orderIsUpdatedForClient`. By doing this less code is repeated (strengthening encapsulation and cohesion) and making the whole code is more easily read.

## 4.2 Task 2: Inheritance vs. Composition Comparison

While this implementation is technically straightforward, the workflow highlights a critical architectural vulnerability: the new class automatically inherits and exposes all other public methods of the superclass. A user can easily circumvent the usage limit by calling inherited methods like `nextLong()` or `nextDouble()` proving that "inheritance breaks encapsulation" by leaking the superclass interface to the rest of the program. By calling the superclass method only when the limit has not been reached, the class fulfills its functional goal but remains tightly coupled to the parent class, inheriting its entire public interface and making it vulnerable to the "fragile base class" problem where encapsulation is effectively broken.

By using composition, the code doesn't automatically expose any of the library's methods; instead only the specific public methods required for the system's needs are manually chosen and written. This design choice fulfills the core requirement for low coupling and high cohesion because the class only provides the "direct possibility" to get to what is absolutely necessary for the application, such as a limited random value generator. By hiding the internal `Random` instances, this composition-based workflow preserves the object's integrity and prevents external actors from bypassing business rules or modifying internal state through inherited methods. This result is a more robust and modular component that does not leak irrelevant methods from the library into the rest of the application, adhering to the project's goal of maintaining a well-designed public interface.

For this specific scenario, object composition is firmly the preferred approach as it aligns with the modern object-oriented principle of favoring composition over inheritance to achieve a well-designed public interface. While inheritance might appear more elegant or "shorter" at the first glance, it creates a rigid, fragile hierarchy that is difficult to maintain as requirements evolve. Composition keeps objects intact and prevents them from "handing out their attributes" or leaking irrelevant internal states to the rest of the system. Ultimately, by wrapping the utility class rather than extending it, a modular, professional component is created that fulfills the workshop's functional needs without sacrificing the architectural integrity of the "Repair Electric Bike" program.

## 4.3 Task 3: Evaluation of Output Testing

One of the bigger headaches when evaluating output, was that it had been demanded that all objects sent to the view had to be printed to the terminal. Since it had also been

requested that all methods returning view should return a `RepairOrderDTO`, there was the problem of how to display the repair order.

If a view had been implemented, different parts of the DTO could have been needed to display the changes. Maybe that a button changed color if that problem had been found in the bike. But since everything had to be displayed, it was decided to print the repair order as an unfinished receipt. This however was not accounted for in the code.

Originally an interface for repair orders had been made so that the `PrintToString` class should be able to take both repair orders and their DTO, but that went against the definition of an DTO, which should not be able to access any code or have the possibility to meddle with the program.

The solution was then to change so that the `RepairOrderDTO` contained a variable `repOrderAsString`. When the DTO is created, the repair order is thus sent in as a string to the DTO, from which the receipt string can then be extracted later.

The other option was, instead of creating another variable in the DTO, to extract the repair Order ID from the DTO and then from that find the corresponding repair order and converting that to a string. The reason it was decided against it, was that this creates would take much longer time and computing power as the repair order would have to be found in the database. At the moment, the database with repair orders is small enough to not have an inpact, and the 'database' is an `ArrayList` where the repair order ID corresponds to the index in the database. However, if a real database is used with another system of ID's, then it could take unnecessary time to get the repair order corresponding to the DTO.

The written output tests for the "Repair Electric Bike" program are evaluated against the explicit benchmarks defined in the course assessment criteria to ensure they meet the standards for professional software verification. These tests are required to be self-evaluating, meaning they must use a framework like JUnit to print an informative message only if a test fails while remaining silent if it passes, which was followed trough out all of the implementation. Following established organizational guidelines, the tests are placed in the same package as the system under test but are located in a separate directory to maintain a clean distinction between production and test code. The criteria also mandate that there is exactly one test class for every tested class, and the suite must be comprehensive enough to cover all branches of if-statements to verify that the system correctly outputs information across various logic paths, such as different repair order states. For each path, a different method was created. This does not mean that there only in one `assert` per method, but that each method only works with one path of testing. The tests that are within the same method are thus related and dependent.

All in all, Automation is achieved through the integration of the JUnit framework, allowing for the rapid and frequent verification of terminal output without the need for manual inspection. The independence of the test suite is preserved by the strict one-to-

one mapping between system classes and test classes, which prevents failures in one area from obscuring issues in another. Finally, because the system is designed to run without external dependencies like real databases or outside hardware—instead using objects in the integration layer to store data—the tests are highly repeatable across different development environments.

# 5 References


- Geekific. (2021a, May 22). The Singleton pattern explained and implemented in Java | Creational Design Patterns | Geekific [Video]. YouTube.

  https://www.youtube.com/watch?v=tSZn4wkBIu8

- Geekific. (2021b, July 10). The template method pattern explained implemented in Java | Behavioral Design Patterns | Geekific [Video]. YouTube.

  https://www.youtube.com/watch?v=cGoVDzHvD4A

- Geekific. (2021c, September 25). The strategy pattern explained and implemented in Java | Behavioral Design Patterns | Geekific [Video]. YouTube.

  https://www.youtube.com/watch?v=Nrwj3gZiuJU

- GeeksforGeeks. (n.d.). Java Interface. GeeksforGeeks.

  http://www.geeksforgeeks.org/java/interfaces-in-java/

- GeeksforGeeks. (2025, July 11). Java current date and time. GeeksforGeeks.

  https://www.geeksforgeeks.org/java/java-current-date-time/

- GeeksforGeeks. (2026, March 13). File handling in Java. GeeksforGeeks.

  https://www.geeksforgeeks.org/java/file-handling-in-java/

- Lindbäck, L. (March 23 2026) *A First Course in Object-Oriented Development: A Hands-On Approach.*

- Marić, P. (2024a, January 8). Testing an Abstract Class With JUnit. Baeldung.

  https://www.baeldung.com/junit-test-abstract-class

- Marić, P. (2024b, January 8). The Observer Pattern in Java. Baeldung.

  https://www.baeldung.com/java-observer-pattern

- Paraschiv, E. (2023, December 1). Java – Write to File. Baeldung.

  https://www.baeldung.com/java-write-to-file

- Software Testing. (2025, May 5). Java Abstract Classes explained simply | Software testing [Video]. YouTube.

  https://www.youtube.com/watch?v=V6iW78WMm2w